

Open Ended Lab
Optimizing Agricultural Satellite Imagery:
Compression, Enhancement, and Feature Extraction
Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

24th Jan, 2025

Department of Computer Engineering,
HITEC University, Taxila

Task No 1:

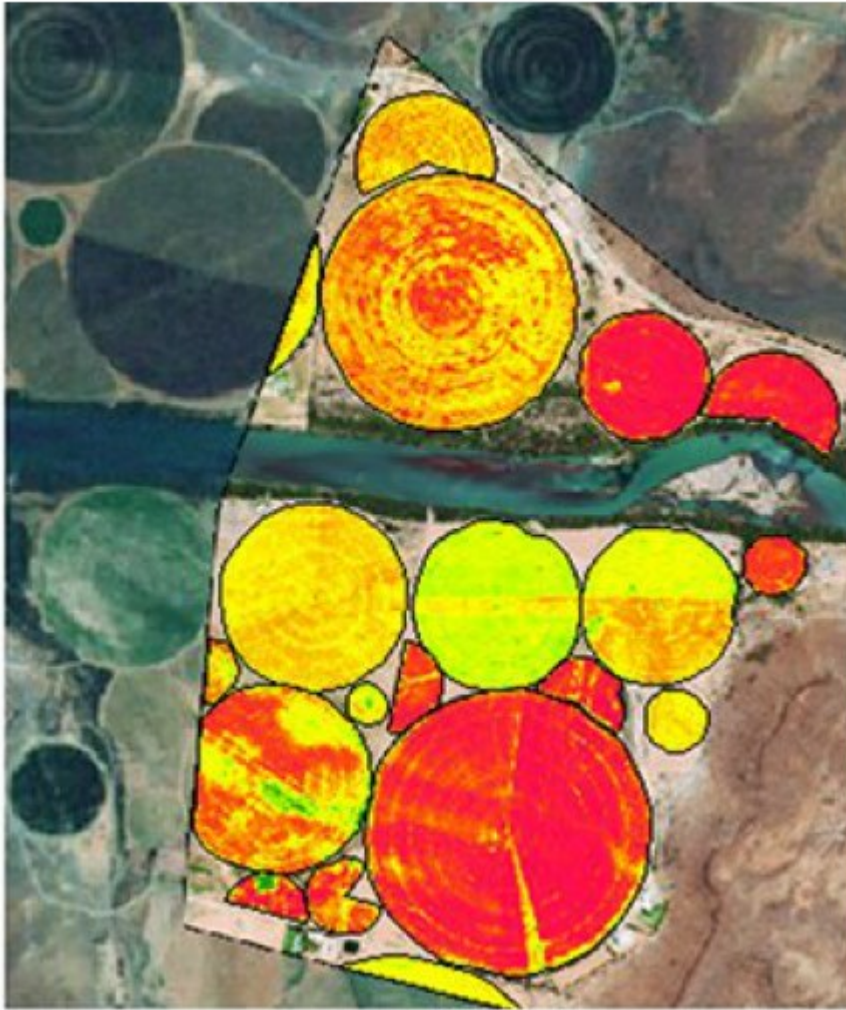
Solution:

Brief description (3-5 lines)
Image Compression 1. Implement and compare compression techniques: Huffman Coding Run-Length Coding (RLC) Calculate and report compression ratios and storage requirements for each method. Provide before-and-after visualizations of compressed satellite images. Compare the quality of original and enhanced images. Highlight improvements in visibility of agricultural features. Provide segmented images with labeled regions (e.g., crops, water, barren land). Analyze the effect of morphological operations on feature extraction. Submit edge-detected images with clear annotations of significant features.
The code <pre>from PIL import Image import matplotlib.pyplot as plt import numpy as np # Load the uploaded image image_path = "OEL-Image-1.jpg" image = Image.open(image_path) # Display the original image plt.figure(figsize=(6, 6)) plt.title("Original Image") plt.imshow(image) plt.axis("off") plt.show() # Convert the image to grayscale for processing gray_image = image.convert("L") # Display the grayscale image</pre>

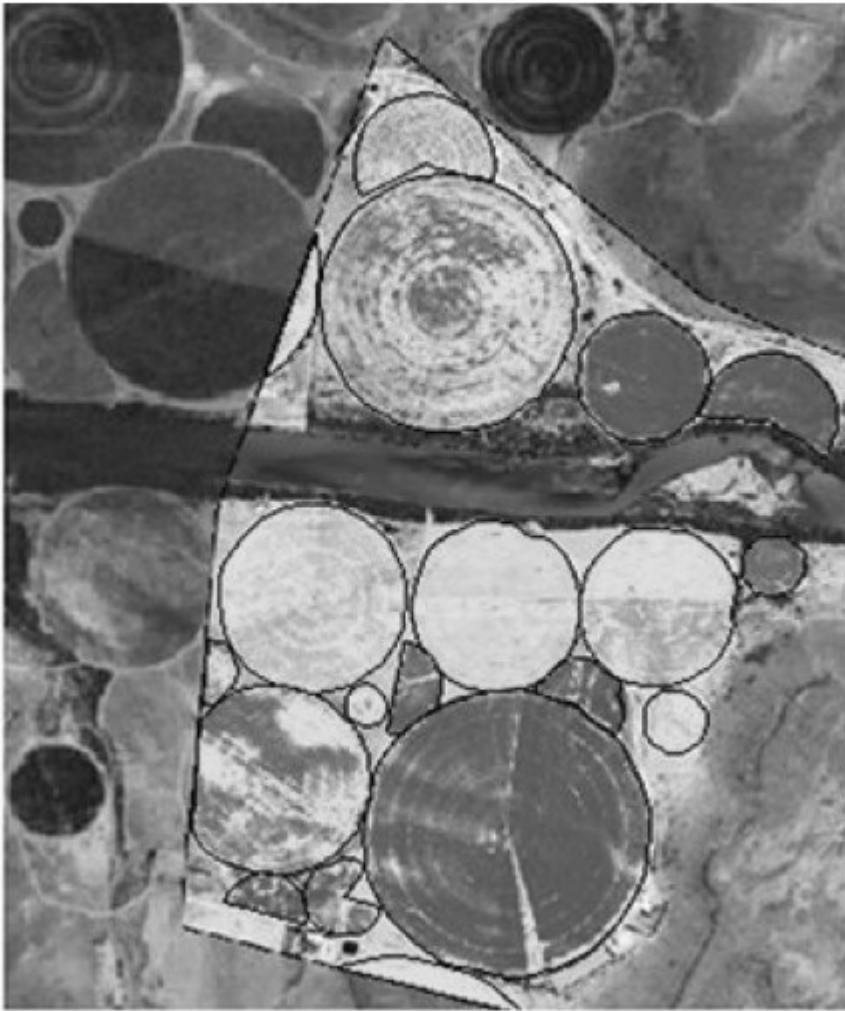
```
plt.figure(figsize=(6, 6))
plt.title("Grayscale Image")
plt.imshow(gray_image, cmap="gray")
plt.axis("off")
plt.show()
# Convert the grayscale image to a NumPy array for further processing
gray_array = np.array(gray_image)
gray_array.shape
```

The results (Screenshot)

Original Image



Grayscale Image



Lab Task No 2:

Solution:

Brief description (3-5 lines)
Image Enhancement 1. Enhance satellite images for better interpretability of agricultural features: - o Apply Histogram Equalization to adjust image contrast. - o Use Smoothing Filters (e.g., Gaussian blur) to reduce noise. - o Experiment with sharpening filters to highlight key details. Deliverable: - o Calculate and report compression ratios and storage requirements for each

method.

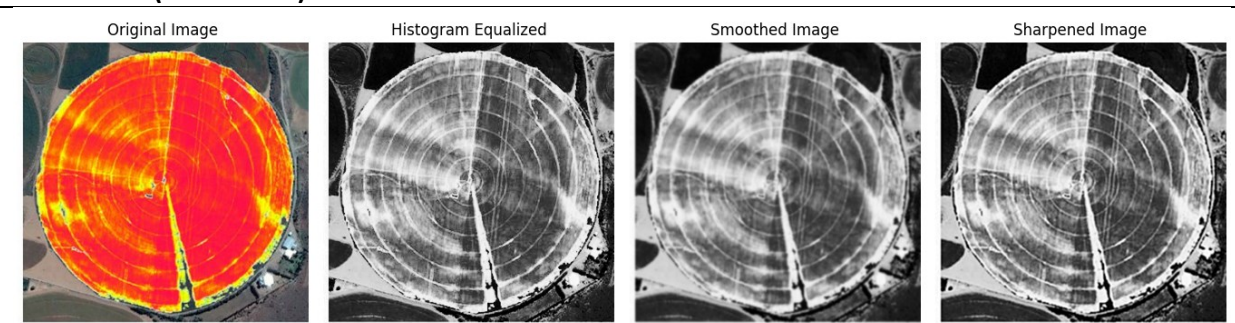
- o Provide before-and-after visualizations of compressed satellite images.
- o Compare the quality of original and enhanced images.
- o Highlight improvements in visibility of agricultural features.
- o Provide segmented images with labeled regions (e.g., crops, water, barren land).
- o Analyze the effect of morphological operations on feature extraction.
- o Submit edge-detected images with clear annotations of significant features.

The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image
image_path = 'OEL-Image-2.jpg' # Path to your uploaded image
image = cv2.imread(image_path)
# Convert the image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Histogram Equalization
hist_eq_image = cv2.equalizeHist(gray_image)
# Apply Gaussian Blur for smoothing
smoothed_image = cv2.GaussianBlur(hist_eq_image, (5, 5), 0)
# Apply a sharpening filter
kernel = np.array([[0, -1, 0],
                   [-1, 5, -1],
                   [0, -1, 0]])
sharpened_image = cv2.filter2D(smoothed_image, -1, kernel)
# Plot the results
plt.figure(figsize=(12, 8))
plt.subplot(1, 4, 1)
plt.title('Original Image')
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.subplot(1, 4, 2)
plt.title('Histogram Equalized')
plt.imshow(hist_eq_image, cmap='gray')
plt.axis('off')
plt.subplot(1, 4, 3)
```

```
plt.title('Smoothed Image')
plt.imshow(smoothed_image, cmap='gray')
plt.axis('off')
plt.subplot(1, 4, 4)
plt.title('Sharpened Image')
plt.imshow(sharpened_image, cmap='gray')
plt.axis('off')
plt.tight_layout()
plt.show()
# Save the processed images
cv2.imwrite('/mnt/data/histogram_equalized.png', hist_eq_image)
cv2.imwrite('/mnt/data/smoothed_image.png', smoothed_image)
cv2.imwrite('/mnt/data/sharpened_image.png', sharpened_image)
```

The results (Screenshot)



Lab Task No 3:

Solution:

Brief description (3-5 lines)

- 1.Feature Segmentation:
 - o Use thresholding to segment areas of interest (e.g., green crop regions, water bodies).
 2. Morphological Operations:
 - o Perform erosion and dilation to clean up noise and enhance feature boundaries.
 - o Apply opening to separate overlapping crop regions.
 - o Use closing to fill gaps in features like irrigation canals.
 - o Experiment with morphological gradient to detect edges of agricultural zones.
- Deliverable:
- o Calculate and report compression ratios and storage requirements for each method.

- o Provide before-and-after visualizations of compressed satellite images.
- o Compare the quality of original and enhanced images.
- o Highlight improvements in visibility of agricultural features.
- o Provide segmented images with labeled regions (e.g., crops, water, barren land).
- o Analyze the effect of morphological operations on feature extraction.
- o Submit edge-detected images with clear annotations of significant features.

The code

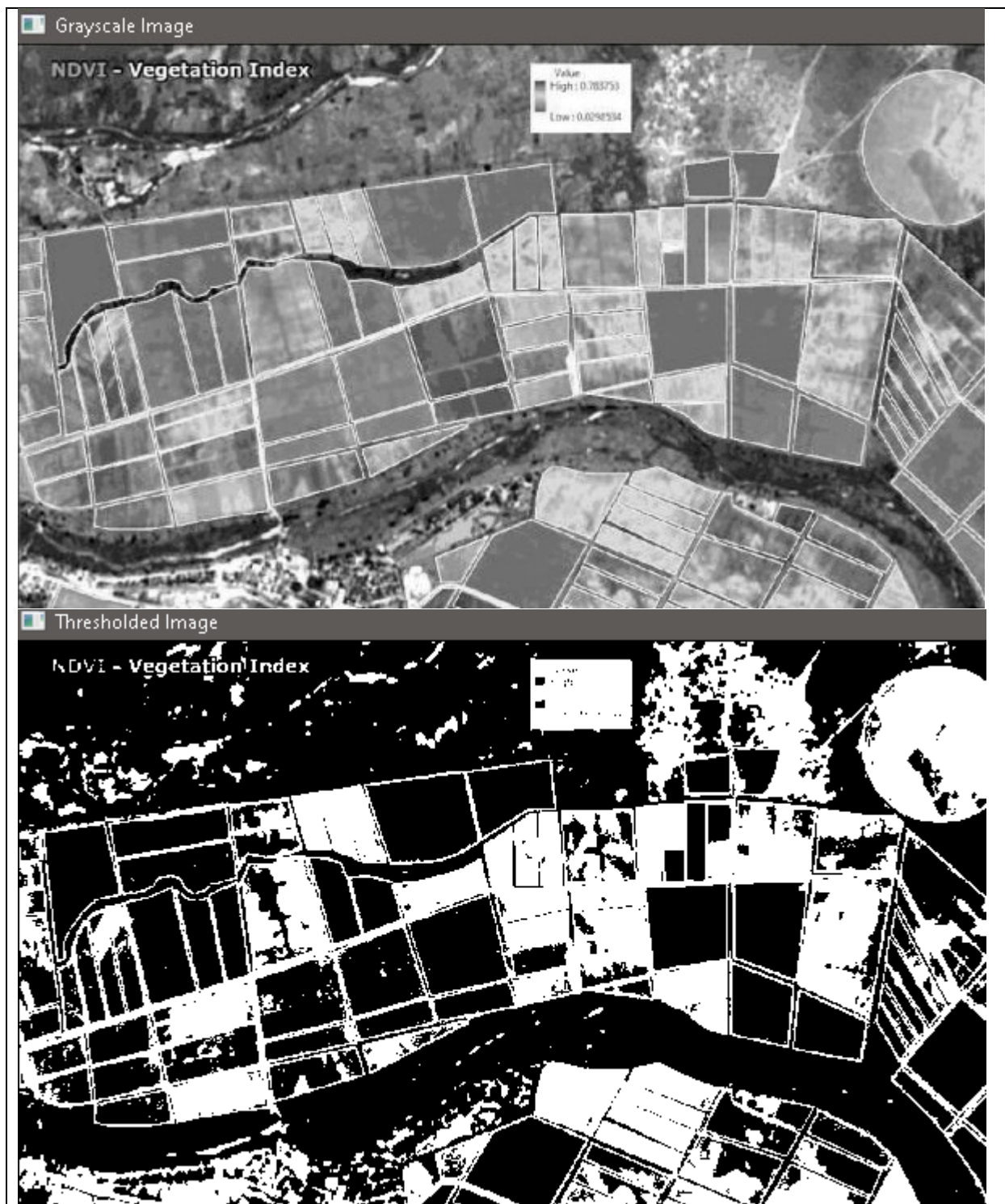
```
import cv2
import numpy as np
# Load the image
image = cv2.imread("OEL-Image-3.jpg")
# Convert the image to grayscale
grayscale_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Otsu's thresholding
_, thresholded_image = cv2.threshold(grayscale_image, 0, 255,
cv2.THRESH_BINARY + cv2.THRESH_OTSU)
# Define a kernel for morphological operations
kernel = np.ones((5, 5), np.uint8) # You can change the size of the
kernel based on requirements
# Perform Erosion
erosion = cv2.erode(thresholded_image, kernel, iterations=1)
# Perform Dilation
dilation = cv2.dilate(thresholded_image, kernel, iterations=1)
# Perform Opening (Erosion followed by Dilation)
opening = cv2.morphologyEx(thresholded_image,
cv2.MORPH_OPEN, kernel)
# Perform Closing (Dilation followed by Erosion)
closing = cv2.morphologyEx(thresholded_image,
cv2.MORPH_CLOSE, kernel)
# Display the results
cv2.imshow("Original Image", image)
cv2.imshow("Grayscale Image", grayscale_image)
cv2.imshow("Thresholded Image", thresholded_image)
cv2.imshow("Erosion", erosion)
cv2.imshow("Dilation", dilation)
cv2.imshow("Opening", opening)
```

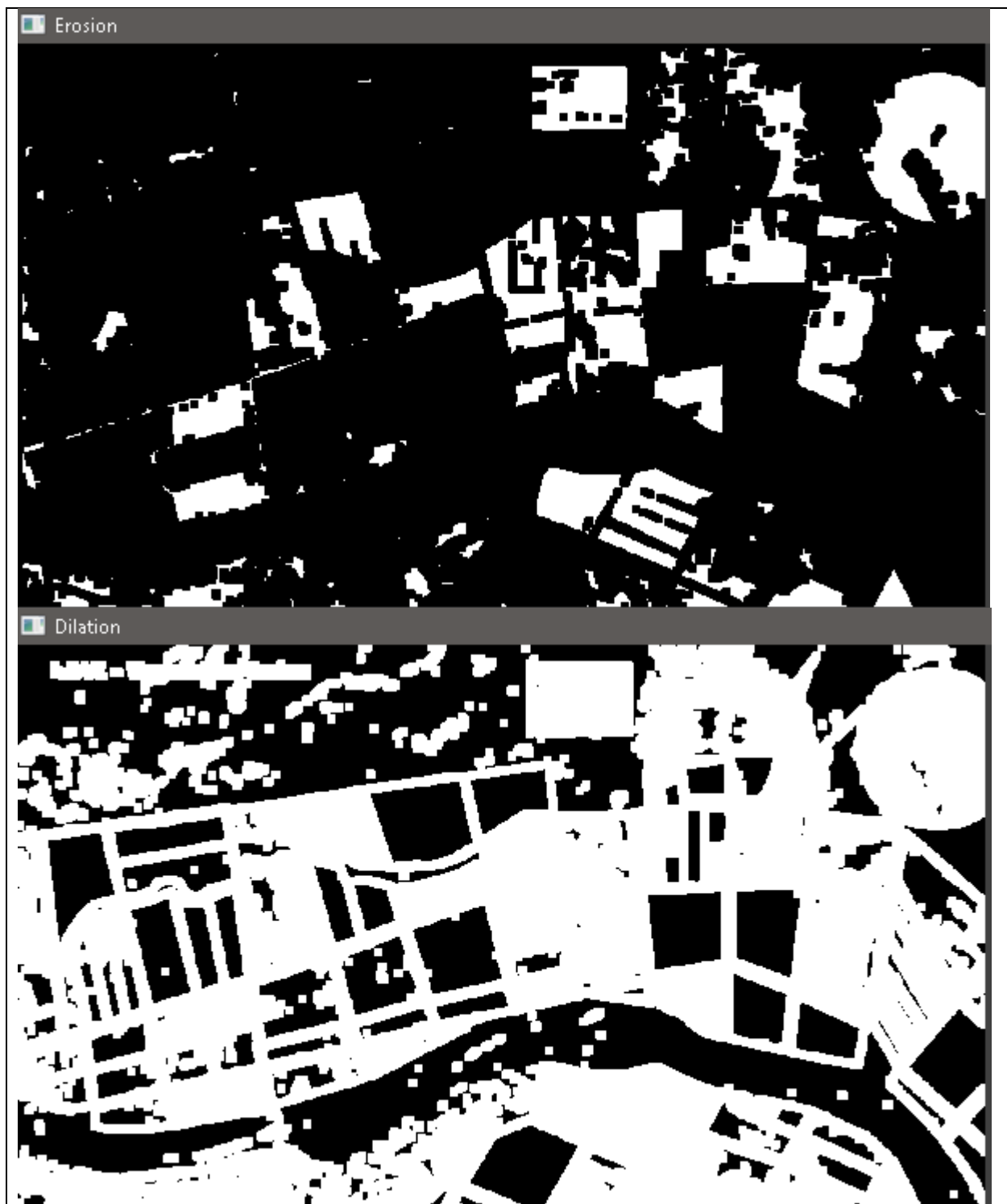


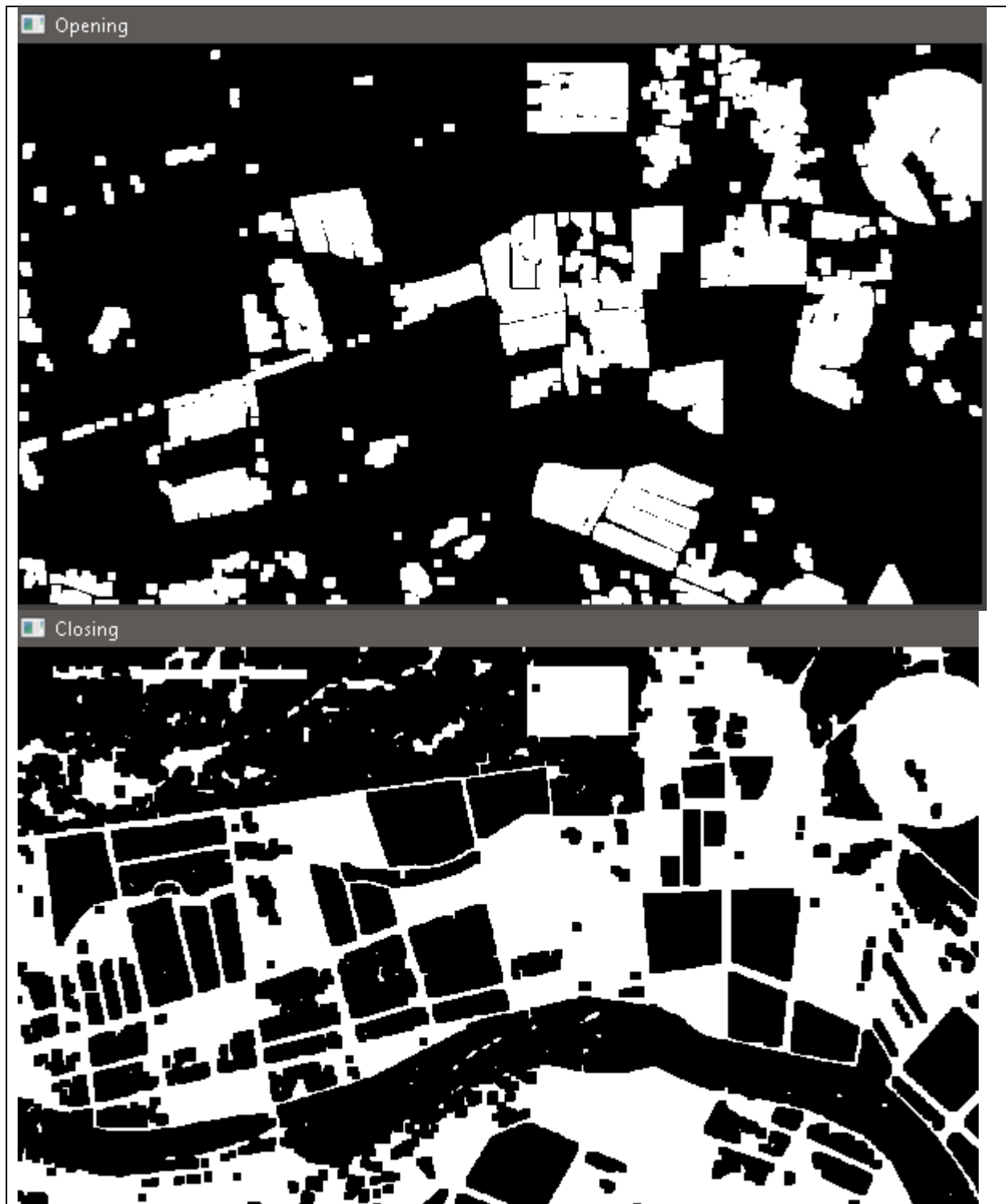
```
cv2.imshow("Closing", closing)
# Wait for a key press and close the windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

The results (Screenshot)









Lab Task No 4:

Solution:

Brief description (3-5 lines)

Edge Detection

1. Apply edge detection algorithms (e.g., Sobel, Canny) to identify field boundaries and irrigation networks.

Deliverable:

- o Calculate and report compression ratios and storage requirements for each method.
- o Provide before-and-after visualizations of compressed satellite images.
- o Compare the quality of original and enhanced images.
- o Highlight improvements in visibility of agricultural features.
- o Provide segmented images with labeled regions (e.g., crops, water, barren land).
- o Analyze the effect of morphological operations on feature extraction.
- o Submit edge-detected images with clear annotations of significant features.

The code

```
from PIL import Image
import matplotlib.pyplot as plt
import cv2
import numpy as np
# Load the uploaded image
image_path = 'OEL-Image-4.jpg'
image = cv2.imread(image_path)
# Convert to grayscale for edge detection
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Sobel Edge Detection
sobel_x = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)
sobel_edge = cv2.magnitude(sobel_x, sobel_y)
# Apply Canny Edge Detection
canny_edges = cv2.Canny(gray_image, 100, 200)
# Display the results
fig, axs = plt.subplots(1, 3, figsize=(15, 5))
# Original Image
axs[0].imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
axs[0].set_title('Original Image')
axs[0].axis('off')
# Sobel Edge Detection
axs[1].imshow(sobel_edge, cmap='gray')
```

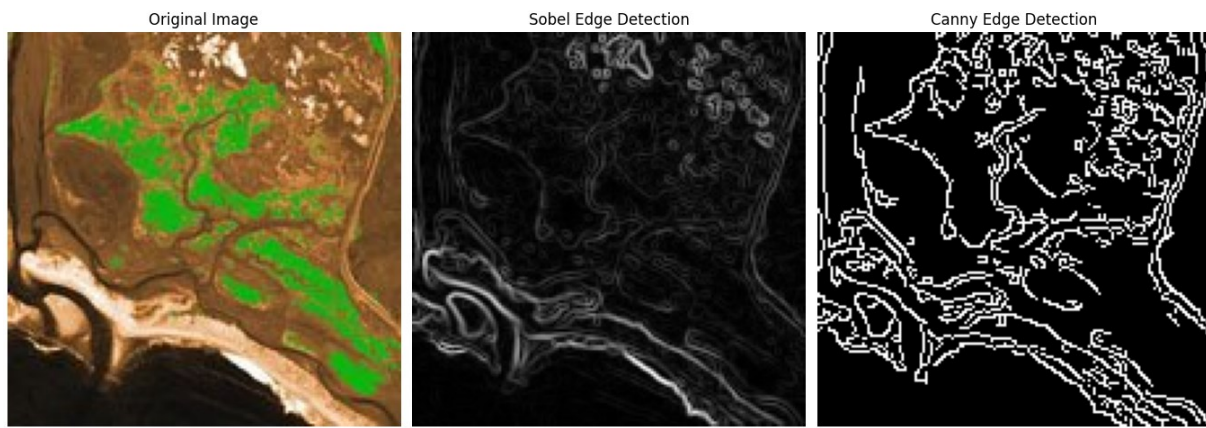


```

axs[1].set_title('Sobel Edge Detection')
axs[1].axis('off')
# Canny Edge Detection
axs[2].imshow(canny_edges, cmap='gray')
axs[2].set_title('Canny Edge Detection')
axs[2].axis('off')
plt.tight_layout()
plt.show()

```

The results (Screenshot)



Conclusion

In conclusion, Optimizing agricultural satellite imagery through compression, enhancement, and feature extraction enables efficient data storage, improved image clarity, and accurate analysis of critical agricultural features. These advancements support better decision-making in crop monitoring, yield prediction, and resource management. Ultimately, the approach enhances precision agriculture and promotes sustainable farming practices.